

Blueprint: AgriVision — Plataforma Unificada de Producción

Generado: Marzo 2026 | Modo: Direct (sin git en canvas) Objetivo: Convertir los módulos Streamlit aislados en una plataforma SaaS multi-tenant desplegada en Hetzner

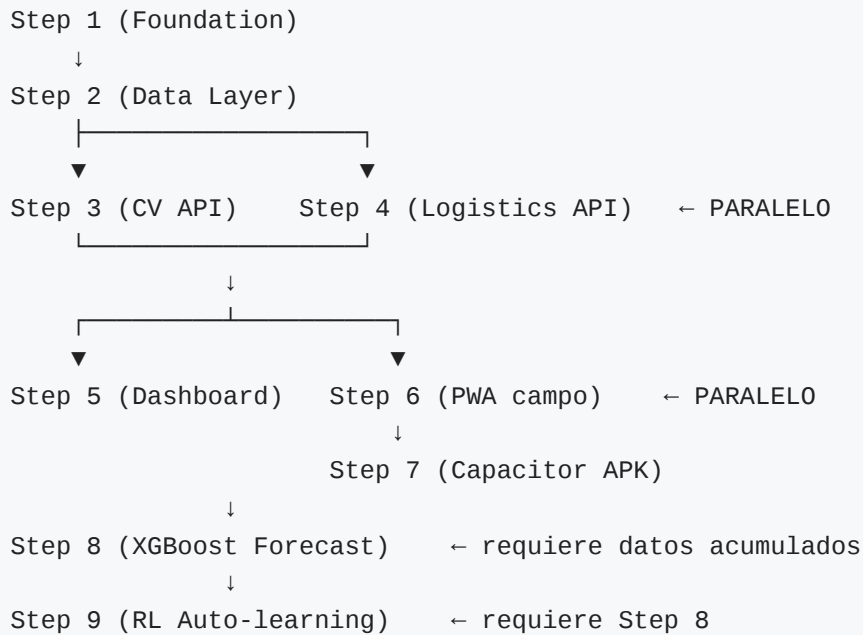
Resumen Ejecutivo

Atributo	Valor
Pasos totales	9
Pasos en paralelo	2 pares (Steps 3+4, Steps 5+6)
Duración estimada	16–24 semanas (1 desarrollador)
Stack principal	FastAPI · PostgreSQL · YOLOv8 · Next.js PWA · Capacitor · Docker · Hetzner
Módulos a integrar	piloto/ · proyecto_conteo_cajas/ · serial_reader_app/
Módulos a construir	XGBoost forecast · RL auto-learning · multi-tenant auth
Training	Google Colab (permanece externo) → modelos deployados a Hetzner

Arquitectura Target



Grafo de Dependencias



Step 1 — Foundation: Monorepo + FastAPI + Docker + Hetzner CI/CD

Context Brief (cold-start)

AgriVision tiene módulos Python/Streamlit aislados en `/home/mauro/personalProjects/agrivision/`. El objetivo es crear una estructura de monorepo unificada con FastAPI como backend, Docker Compose para orquestación local, y despliegue automático en el VPS de Hetzner que ya existe en otro proyecto. No hay git en este directorio todavía.

Estado actual

- Módulos existentes: `piloto/`, `proyecto_conteo_cajas/`, `serial_reader_app/`, `qr_cajas/`, `clavel/`
- Todos corren en local con Streamlit
- Hetzner VPS ya disponible (en uso para otro proyecto)
- Sin API unificada, sin Docker, sin CI/CD

Tareas

- ☐ **1.1** Crear estructura de monorepo:

```
agrivision-platform/
├── backend/
│   ├── app/
│   │   ├── main.py          # FastAPI entry point
│   │   ├── api/v1/          # Routers por módulo
│   │   ├── core/            # Config, auth, settings
│   │   ├── models/          # SQLAlchemy models
│   │   └── services/        # Business logic
│   ├── Dockerfile
│   └── requirements.txt
├── frontend/                # Step 5 – Next.js
├── ml/
│   ├── cv_clavel/           # Migrado de piloto/
│   ├── logistics/           # Migrado de proyecto_conteo_cajas/
│   └── models/              # Modelos .pt deployados
├── docker-compose.yml
├── docker-compose.prod.yml
└── .github/workflows/deploy.yml
```

- ☐ **1.2** FastAPI skeleton: `main.py` con health check `/api/v1/health`, CORS, lifespan
- ☐ **1.3** Docker Compose local: servicios `api`, `db` (postgres:15), `redis`, `minio`
- ☐ **1.4** Nginx config en Hetzner: reverse proxy → FastAPI (puerto 8000), TLS con Let's Encrypt
- ☐ **1.5** GitHub Actions workflow:
 - Push a `main` → build Docker image → SSH a Hetzner → `docker compose pull && docker compose up -d`
- ☐ **1.6** Variables de entorno: `.env.example` con todas las variables documentadas
- ☐ **1.7** `pyproject.toml` con dependencias base: `fastapi`, `uvicorn[standard]`, `sqlalchemy`, `alembic`, `pydantic-settings`, `python-jose`, `passlib`, `ultralytics`, `redis`, `boto3` (para MinIO)

Verificación

```
# Local
docker compose up -d
curl http://localhost:8000/api/v1/health # → {"status": "ok"}

# Producción
curl https://api.agrivision.co/api/v1/health # → {"status": "ok"}
```

Exit criteria

- FastAPI responde en local y en Hetzner
- CI/CD: push a main despliega en < 3 min
- Docker Compose levanta los 4 servicios sin errores

Rollback

```
# En Hetzner: revertir a imagen anterior
docker compose down
docker compose up -d --no-recreate
```

Tiempo estimado: 1–2 semanas

Step 2 — Data Layer: PostgreSQL Multi-tenant + MinIO + Redis

Context Brief (cold-start)

Step 1 completado. FastAPI corre en Hetzner. Necesitamos el esquema de datos multi-tenant para soportar múltiples floricultoras como clientes independientes. Cada finca tiene lotes, camas, registros de CV y datos de logística. MinIO (S3-compatible, self-hosted en Hetzner) almacena las imágenes de campo. Redis cachea los resultados de inferencia.

Tareas

- ☐ **2.1** Diseño del esquema multi-tenant:

```

-- Tenants (floricultoras)
tenants: id, name, slug, plan, created_at
users: id, tenant_id, email, role (admin/manager/field_worker), hashed_password

-- Estructura del cultivo
fincas: id, tenant_id, name, location, total_ha
lotes: id, finca_id, name, crop_type (clavel/rosa/...), ha
camas: id, lote_id, name, plants_count

-- Registros CV (campo)
cv_registros: id, cama_id, user_id, image_path, captured_at,
               estado_0_count, estado_1_count, estado_2_count, estado_3_count,
               disease_detected (bool), disease_type, confidence

-- Inventario consolidado (agregado diario)
inventario_diario: id, lote_id, date,
                  total_estado_0, total_estado_1, total_estado_2, total_estado_3

-- Logística
despachos: id, tenant_id, date, camion_id, total_cajas, status
pallets: id, despacho_id, pallet_number, cajas_count
qr_registros: id, pallet_id, caja_qr_code, scanned_at, scanned_by

-- Histórico de producción (para XGBoost – Step 8)
produccion_historica: id, lote_id, fecha_corte, cantidad_tallos, variedad

```

- ☐ **2.2 Alembic:** migrations iniciales para todas las tablas
- ☐ **2.3 SQLAlchemy models** con relaciones ORM
- ☐ **2.4 MinIO setup** en Hetzner: bucket `agrivision-images` con política por tenant
- ☐ **2.5 Utilidad de upload:** `services/storage.py` → imagen → MinIO → retorna URL firmada
- ☐ **2.6 Redis config:** cache de predicciones CV por 10 min (evitar re-inferencia de la misma imagen)
- ☐ **2.7 JWT auth completo:**
 - `POST /api/v1/auth/login` → access token + refresh token
 - Middleware de tenant isolation: cada request lleva `tenant_id` en el token
 - Roles: `admin`, `manager`, `field_worker`
- ☐ **2.8 Script seed:** crear tenant demo + usuario admin para pruebas

Verificación

```
# Migrations
alembic upgrade head

# Auth
curl -X POST /api/v1/auth/login \
  -d '{"email":"admin@demo.com","password":"demo123"}' \
  # → {"access_token": "...", "token_type": "bearer"}

# Isolation check: token de tenant A no puede ver datos de tenant B
```

Exit criteria

- Todas las migrations corren sin errores
- JWT funciona con tenant isolation
- MinIO almacena y sirve imágenes
- Redis cachea predicciones

Tiempo estimado: 1–2 semanas

Step 3 — CV Service: Clavel Detection API 🔄 (paralelo con Step 4)

Context Brief (cold-start)

Steps 1+2 completados. El módulo `piloto/` tiene un pipeline YOLOv8 con 4 clases de estados fisiológicos del clavel (Temprano1, Temprano2, Medio, Maduro), 312 imágenes con augmentation, listo para re-entrenar. El modelo actual (`yolo8n.pt`) necesita re-entrenamiento con las 4 clases consolidadas (`clavel-vision/data/final_dataset/data.yaml`). Se debe exponer como endpoint FastAPI que recibe imagen → retorna detecciones + conteos por estado.

Estado actual en `piloto/`

- Dataset: 312 imgs, 20,608 bboxes, 4 clases, `data.yaml` listo
- Apps Streamlit: `app_clavel.py`, `app_conteo_cama.py`, `app_conteo_video.py`
- Modelo pendiente de re-entrenamiento con 4 clases nuevas
- Documentación de entrenamiento incremental completa

Tareas

- ☐ **3.1** Re-entrenar modelo en Google Colab:

```
# En Colab (ver PASO_A_PASO_ENTRENAMIENTO.md)
model = YOLO('yolov8n.pt')
results = model.train(
    data='final_dataset/data.yaml',
    epochs=100, imgsz=1280, batch=8, patience=20,
    name='clavel_4classes_v1'
)
# Target: mAP@0.5 > 0.85
```

- ☐ **3.2** Copiar modelo entrenado a `ml/cv_clavel/models/clavel_v1.pt`
- ☐ **3.3** `ml/cv_clavel/inference.py`: clase `ClavellInference`

```
class ClavellInference:
    def detect(self, image_bytes) -> ClavellResult:
        # Retorna: {estado_0: int, estado_1: int, estado_2: int, estado_3: int,
        #           disease_detected: bool, disease_type: str, confidence: float,
        #           annotated_image_b64: str}
```

- ☐ **3.4** Router FastAPI `api/v1/cv.py`:

```
POST /api/v1/cv/detect
  Body: multipart/form-data (image, cama_id)
  Auth: JWT (field_worker+)
  Returns: ClavellResult + registro guardado en DB + imagen en MinIO

GET /api/v1/cv/inventory/{lote_id}
  Returns: inventario consolidado del lote por estado

GET /api/v1/cv/alerts/{finca_id}
  Returns: alertas de enfermedad de las últimas 24h
```

- ☐ **3.5** Background task: al recibir detección → guardar en `cv_registros` + actualizar `inventario_diario`
- ☐ **3.6** Tests de integración: imagen de prueba → endpoint → `assert counts > 0`
- ☐ **3.7** Migrar lógica útil de `app_clavel.py` y `conteo_cama_utils.py` al servicio

Verificación

```
curl -X POST /api/v1/cv/detect \
  -H "Authorization: Bearer $TOKEN" \
  -F "image=@test_clavel.jpg" \
  -F "cama_id=1"
# → {"estado_0": 12, "estado_1": 45, "estado_2": 23, "estado_3": 8,
#    "disease_detected": false, ...}
```

Exit criteria

- Modelo re-entrenado con **mAP@0.5** > 0.85
- Endpoint responde en < 3 segundos en Hetzner
- Resultados guardados en DB + imagen en MinIO

Tiempo estimado: 2–3 semanas

Step 4 — Logistics Service: Conteo de Cajas + QR API ↔ (paralelo con Step 3)

Context Brief (cold-start)

Steps 1+2 completados. Dos módulos listos para integrar:

1. `proyecto_conteo_cajas/` : YOLOv8 (97.45% mAP50), 3 clases (pallet/lateral/frontal), modelo `best.pt` en producción
2. `serial_reader_app/` : lectura de QR y barcodes, múltiples técnicas (OpenCV + CLAHE + rotación), MCP server existente Ambos se exponen como endpoints FastAPI para que la app móvil pueda enviar fotos y recibir datos de logística.

Estado actual

- `proyecto_conteo_cajas/models/best_model_3classes_v2/best.pt` → 97.45% mAP50, LISTO
- `serial_reader_app/app.py` → lectura QR/barcode con Streamlit, lógica reutilizable
- Sin unificación, sin persistencia, sin multi-tenant

Tareas

- ☐ **4.1** Copiar modelo a `ml/logistics/models/conteo_cajas_v2.pt`

- ☐ **4.2** `ml/logistics/box_counter.py` : extraer lógica de `inference_roi.py`

```
class BoxCounter:
    def count(self, image_bytes, view: Literal['frontal', 'lateral']) -> BoxResult:
        # Returns: {pallet_detected: bool, frontal_count: int, lateral_count: int}
```

- ☐ **4.3** `ml/logistics/qr_scanner.py` : extraer mejor lógica de `serial_reader_app/`

```
class QRScanner:
    def scan(self, image_bytes) -> list[QRResult]:
        # Returns: [{code: str, type: 'QR'|'barcode', confidence: float}]
```

- ☐ **4.4** Router `api/v1/logistics.py` :

```
POST /api/v1/logistics/count-boxes
  Body: {image_frontal: file, image_lateral: file, despacho_id: int}
  Returns: {total_cajas: int, frontal: int, lateral: int, annotated_images: [...]}

POST /api/v1/logistics/scan-qr
  Body: {image: file, pallet_id: int}
  Returns: {codes_found: [{code, type}], registered: int}

POST /api/v1/logistics/create-pallet
  Body: {despacho_id, expected_boxes}
  Returns: {pallet_id, pallet_number}

GET /api/v1/logistics/despacho/{id}
  Returns: resumen completo: pallets, cajas, camión, estado

GET /api/v1/logistics/dashboard/{tenant_id}
  Returns: inventario de despachos del día
```

- ☐ **4.5** Persistencia: al escanear QR → guardar en `qr_registros` + actualizar `pallets.cajas_count`
- ☐ **4.6** Regla de negocio: alerta si pallet supera capacidad esperada
- ☐ **4.7** Tests: imagen de pallet real → `assert count > 0`

Verificación

```
# Conteo de cajas
curl -X POST /api/v1/logistics/count-boxes \
  -H "Authorization: Bearer $TOKEN" \
  -F "image_frontal=@pallet_front.jpg" \
  -F "image_lateral=@pallet_side.jpg" \
  -F "despacho_id=1"
# → {"total_cajas": 48, "frontal": 24, "lateral": 24}

# Escaneo QR
curl -X POST /api/v1/logistics/scan-qr \
  -F "image=@caja_qr.jpg" \
  -F "pallet_id=3"
# → {"codes_found": [{"code": "C0-123456", "type": "QR"}], "registered": 1}
```

Exit criteria

- Conteo de cajas con precisión > 90% en imágenes de prueba reales
- QR scan con detección > 95% en condiciones normales de campo
- Datos persistidos correctamente en DB

Tiempo estimado: 2 semanas

Step 5 — Dashboard Web: Next.js + Auth + Vistas Gerenciales

(paralelo con Step 6)

Context Brief (cold-start)

Steps 1-4 completados. FastAPI sirve CV y logística. Necesitamos un dashboard web para gerentes y admins. Next.js (App Router) con Tailwind CSS. Consume la API FastAPI. Autenticación JWT con cookies. Multi-tenant: cada usuario solo ve datos de su floricultora.

Tareas

- ☐ **5.1** Setup frontend/ con Next.js 15 + Tailwind CSS + shadcn/ui
- ☐ **5.2** Auth flow: login → JWT en cookie httpOnly → middleware de rutas protegidas
- ☐ **5.3** Vista: **Inventario Fisiológico del Lote**
 - Tabla: cama × estado (Estado 0 / 1 / 2 / 3)
 - Gráfico de barras apiladas: evolución semanal por estado

- Indicador: % flores en estado Maduro (listas para corte)
- ☐ **5.4 Vista: Alertas de Enfermedad**
 - Lista: camas con detección positiva + timestamp + imagen anotada
 - Badge rojo si > 10% plantas afectadas en una cama
- ☐ **5.5 Vista: Dashboard Logístico**
 - Despachos del día: pallets armados, cajas totales, camiones
 - Estado por despacho: en proceso / completado / despachado
- ☐ **5.6 Vista: Gerencia General (home)**
 - Cards: total flores por estado hoy, alertas activas, despachos pendientes
 - "Todo a un click" — el dato más urgente primero
- ☐ **5.7 API client (`lib/api.ts`)**: wrapper tipado para todos los endpoints FastAPI
- ☐ **5.8 Deploy frontend en Hetzner** (Nginx sirve el build estático de Next.js)

Verificación

- Gerente puede ver inventario por estado actualizado al minuto
- Alerta de enfermedad aparece en < 60 seg después de que el técnico sube una foto
- Dashboard funciona en desktop y tablet

Exit criteria

- 4 vistas funcionales consumiendo datos reales de la API
- Auth multi-tenant: usuario de finca A no ve datos de finca B
- Responsive en tablet (1024px+)

Tiempo estimado: 3 semanas

Step 6 — PWA Campo: Captura Móvil para Técnicos 🔄 (paralelo con Step 5)

Context Brief (cold-start)

Steps 1-4 completados. Los técnicos de campo necesitan capturar fotos del cultivo y escanear QR desde Android. La estrategia es PWA primero (funciona en browser Android sin instalar nada) → luego Capacitor para APK. Esta es la ruta de menor esfuerzo para tener presencia en Android sin desarrollar app nativa desde cero.

Tareas

- ☐ **6.1** Extensión mobile del frontend Next.js (misma codebase, ruta `/campo/`)
- ☐ **6.2** Manifiesto PWA (`manifest.json`): icono, nombre, `display: standalone`, `start_url: /campo`
- ☐ **6.3** Service Worker: caché offline de la UI + cola de fotos pendientes de subir

```
// Si sin conexión: foto queda en IndexedDB
// Al recuperar conexión: sync automático con la API
```

- ☐ **6.4** Vista: **Captura de Cama** (flujo principal del técnico)

```
1. Seleccionar: Finca → Lote → Cama
2. Abrir cámara (Web Camera API)
3. Tomar foto → preview inmediato
4. Enviar → spinner → mostrar resultado:
   "Estado 0: 12 | Estado 1: 45 | Estado 2: 23 | Estado 3: 8"
   + Alerta si enfermedad detectada (badge rojo)
5. Confirmar o rehacer
```

- ☐ **6.5** Vista: **Escaneo QR** (flujo logística)

```
1. Seleccionar pallet
2. Escanear QR con cámara (jsQR o ZXing-js)
3. Código registrado → feedback visual + sonido
4. Contador de cajas del pallet actualizado en tiempo real
```

- ☐ **6.6** UX para campo: botones grandes (60px+), alto contraste, modo "guantes" (tap areas amplias)
- ☐ **6.7** Optimizar imágenes antes de enviar: redimensionar a 1280px max, JPEG 80% (reducir tiempo de upload en campo con 3G/4G)
- ☐ **6.8** Test en Android: abrir URL en Chrome Android → "Añadir a pantalla de inicio" → funciona como app

Verificación

```
1. Técnico abre https://campo.agrivation.co en Chrome Android
2. Añade a pantalla de inicio
3. Abre sin internet → ve UI con cola de pendientes
4. Con internet → sube fotos → recibe resultados en < 5 seg
```

Exit criteria

- PWA instalable en Android desde Chrome
- Funciona offline (UI disponible, fotos en cola)
- Tiempo foto → resultado < 5 segundos con 4G

Tiempo estimado: 2 semanas

Step 7 — Capacitor APK: Empaquetado Android

Context Brief (cold-start)

Step 6 completado. La PWA funciona perfectamente en Android Chrome. Capacitor permite empaquetar la web app como APK nativo con acceso a la cámara nativa (mejor que Web Camera API) y distribución por APK directo (sin Play Store en fase inicial). Solo requiere ajustes mínimos sobre el código web existente.

Tareas

- ☐ 7.1 Instalar Capacitor en el frontend:

```
npm install @capacitor/core @capacitor/cli
npm install @capacitor/camera @capacitor/filesystem @capacitor/network
npx cap init "AgriVision Campo" "co.agrivision.campo"
npx cap add android
```

- ☐ 7.2 Reemplazar Web Camera API con Capacitor Camera plugin (mejor calidad en Android)

```
import { Camera, CameraResultType } from '@capacitor/camera';
const photo = await Camera.getPhoto({
  resultType: CameraResultType.Base64,
  quality: 80, width: 1280
});
```

- ☐ 7.3 Reemplazar IndexedDB offline sync con Capacitor Filesystem (más robusto en Android)
- ☐ 7.4 Network plugin: detectar estado de conexión para activar sync automático
- ☐ 7.5 Build APK:

```
npm run build
npx cap sync android
cd android && ./gradlew assembleRelease
```

- ☐ **7.6** Firmar APK para distribución directa (sin Play Store)
- ☐ **7.7** Script de distribución: APK por WhatsApp/link directo a técnicos

Verificación

- APK instala sin errores en Android 10+
- Cámara nativa abre y captura correctamente
- Resultados de CV visibles en < 5 seg post-captura

Exit criteria

- APK firmado funcionando en mínimo 2 dispositivos Android de prueba
- Cámara nativa con mejor calidad que PWA
- Proceso de update: nuevo APK en < 30 min tras merge a main

Tiempo estimado: 1 semana

Step 8 — XGBoost Forecast Service: Proyección de Producción

Context Brief (cold-start)

Steps 1-7 completados. La plataforma opera y acumula datos de CV diariamente en `cv_registros` e `inventario_diario`. Para este step se necesitan también datos históricos externos (registros de producción de corte anteriores al despliegue del sistema). Step 8 no puede iniciar hasta tener mínimo 4–8 semanas de datos acumulados en producción + import de histórico del cliente.

Pre-requisitos (bloquean este step)

1. ☐ Mínimo 30 días de datos CV en producción (se acumula automáticamente desde Steps 3-6)
2. ☐ Import de datos históricos del cliente (registros de producción anteriores en Excel/CSV)
3. ☐ Definir las variables del modelo con el primer cliente

Tareas

- ☐ **8.1** Script de import de histórico:

```
# Acepta Excel del cliente con columnas:  
# fecha | lote | variedad | tallos_cortados | estado_mercado  
python scripts/import_historical_data.py --file cliente_historico.xlsx --finca_id 1
```

- ☐ **8.2** Feature engineering pipeline (`ml/forecast/features.py`):

```
Features por lote por día:  
- estado_0_ratio, estado_1_ratio, estado_2_ratio, estado_3_ratio  
- dias_desde_ultimo_corte  
- temperatura_promedio (si se integra sensor/API meteorológica)  
- semana_del_año (estacionalidad)  
- lag_7d, lag_14d (estado de hace 1-2 semanas)  
Target: tallos_cortados en t+7 días, t+14 días
```

- ☐ **8.3** Training notebook (`ml/forecast/train_xgboost.ipynb`):
 - Entrenamiento en Google Colab (igual que YOLOv8)
 - Cross-validation temporal (TimeSeriesSplit)
 - Métricas: MAPE, MAE por semana de horizonte
 - Target: MAPE < 20% en semana 1, < 25% en semana 2
- ☐ **8.4** Serializar modelo: `ml/forecast/models/xgboost_lote_{id}_v1.json`
- ☐ **8.5** Router `api/v1/forecast.py` :

```
GET /api/v1/forecast/{lote_id}?weeks=4  
Returns: [{week: 1, predicted_stems: 12400, ci_low: 10800, ci_high: 14000},  
          {week: 2, predicted_stems: 9800, ...}, ...]  
  
POST /api/v1/forecast/actual  
Body: {lote_id, date, actual_stems} ← registrar corte real para reentrenamiento
```

- ☐ **8.6** Vista en dashboard: **Proyección de Producción**
 - Gráfico línea: semanas futuras con banda de confianza
 - Alerta: "La semana 3 muestra déficit vs. pedido comprometido"
 - Botón: "Exportar proyección a CSV" para compartir con cliente en Miami
- ☐ **8.7** Pipeline de reentrenamiento semanal:
 - Cron job cada viernes: toma datos de la semana → refit del modelo → deploy automático

Verificación

```
curl /api/v1/forecast/1?weeks=4 \  
  -H "Authorization: Bearer $TOKEN"  
# → [{week:1, predicted: 12400, ci_low: 10800, ci_high: 14000}, ...]
```

Exit criteria

- MAPE < 20% en primeras 2 semanas de horizonte (validado contra datos históricos)
- Reentrenamiento semanal automático funcionando
- Vista en dashboard muestra proyección con banda de confianza

⚠ Nota crítica

Este step requiere datos. No iniciar hasta tener mínimo 30 días de datos CV + histórico importado. Intentar entrenarlo antes resultará en modelo con alta varianza y error > 40%.

Tiempo estimado: 3–4 semanas (después de datos acumulados)

Step 9 — RL Auto-learning: Agente de Optimización (P5)

Context Brief (cold-start)

Steps 1-8 completados. El sistema tiene meses de datos de campo, proyecciones XGBoost calibradas y registros de cortes reales vs. predichos. El agente de Reinforcement Learning aprende a recomendar cuándo cortar, qué lotes priorizar y qué tratamientos fitosanitarios aplicar para maximizar la producción exportable. Es el componente de mayor complejidad técnica y el que más datos requiere.

Pre-requisitos (bloquean este step)

1. ☐ Mínimo 6 meses de datos de CV en producción
2. ☐ XGBoost calibrado y funcionando (Step 8)
3. ☐ Registros reales de decisiones y resultados (corte real vs. recomendado)

Tareas

- ☐ **9.1** Definir el MDP (Markov Decision Process):

```

Estado (State):
- inventario_fisiologico por lote (4 estados como %)
- disease_alerts activos
- dias_hasta_temporada_alta (San Valentín, etc.)
- pedidos_comprometidos (tallos)
- forecast_xgboost (próximas 4 semanas)

Acciones (Actions):
- cortar_lote(lote_id, porcentaje)
- aplicar_tratamiento(lote_id, tipo_enfermedad)
- priorizar_riego(lote_id)
- no_accion()

Recompensa (Reward):
- +R: tallos exportados / pedido comprometido (cumplimiento)
- -R: tallos descartados × costo_unitario
- -R: enfermedad_propagada × costo_lote
- +R: eficiencia de insumos (tratamiento preciso vs. masivo)

```

- ☐ **9.2 Environment Gym** (`ml/r1/environment.py`):
 - Simular usando datos históricos como ground truth
 - `step(action)` → `next_state`, `reward`, `done`
- ☐ **9.3 Algoritmo RL: PPO (Proximal Policy Optimization) con Stable-Baselines3**

```

from stable_baselines3 import PPO
model = PPO("MlpPolicy", env, verbose=1)
model.learn(total_timesteps=1_000_000)

```

- ☐ **9.4 Training en Google Colab con datos históricos del simulador**
- ☐ **9.5 Router** `api/v1/r1.py`:

```

GET /api/v1/r1/recommendations/{finca_id}
Returns: [{action: "cortar_lote", lote_id: 3, confidence: 0.87,
          reason: "Estado 3 al 65%, pedido de 8000 tallos en 5 días"}, ...]

POST /api/v1/r1/feedback
Body: {recommendation_id, executed: bool, outcome}
← cierra el loop: el agente aprende de lo que el gerente hizo

```

- ☐ **9.6 Vista en dashboard: Recomendaciones Inteligentes**
 - Cards de acción sugerida con justificación
 - Botón "Ejecutar" (registra feedback positivo) o "Ignorar" (feedback negativo)
 - Score de confianza visible
- ☐ **9.7 Retraining pipeline mensual con nuevos datos de feedback**

Exit criteria

- Agente supera baseline (XGBoost solo) en simulación por > 10%
- Recomendaciones tienen justificación legible (explainability básico)
- Loop de feedback cierra correctamente

⚠ Nota crítica

Este es el paso más especulativo. Requiere 6+ meses de datos reales y posiblemente ajuste fino del espacio de estados/acciones. Priorizar datos de calidad y loop de feedback antes que el algoritmo RL en sí.

Tiempo estimado: 4–6 semanas (sujeto a calidad de datos)

Cronograma Orientativo

```
Semana 1- 2: Step 1 – Foundation (FastAPI + Docker + Hetzner CI/CD)
Semana 3- 4: Step 2 – Data Layer (PostgreSQL + MinIO + Redis + Auth)
Semana 5- 7: Step 3 – CV Service  ↵ PARALELO
Semana 5- 6: Step 4 – Logistics   ↵
Semana 8-10: Step 5 – Dashboard Web ↵ PARALELO
Semana 8- 9: Step 6 – PWA Campo   ↵
Semana 11   : Step 7 – Capacitor APK
Semana 12-16: [Acumulación de datos en producción – no hay step técnico activo]
               ↑ Este tiempo = conseguir primer cliente pagante
Semana 17-20: Step 8 – XGBoost Forecast (cuando haya ≥30 días de datos)
Semana 21-26: Step 9 – RL Auto-learning (cuando haya ≥6 meses de datos)
```

Decisiones de Arquitectura (ADRs)

ADR-001: PWA → Capacitor en lugar de React Native

Decisión: Construir PWA primero, empaquetar con Capacitor para Android. **Razón:** Mismo codebase JavaScript para web + mobile. React Native requeriría una segunda codebase. Con Capacitor se reutiliza el 95% del código web. **Trade-off:** La UI no es 100% nativa (pero es suficiente para uso en campo).

ADR-002: YOLOv8n (nano) como modelo de inferencia

Decisión: Mantener YOLOv8n en producción. **Razón:** Tiempo de inferencia <500ms en CPU. El VPS de Hetzner no tiene GPU. YOLOv8m o large requieren GPU para latencia aceptable. **Alternativa si se necesita más accuracy:** YOLOv8s (small) con benchmarking en Hetzner antes de decidir.

ADR-003: Entrenamiento en Google Colab, inferencia en Hetzner

Decisión: Training siempre en Colab (GPU gratuita), modelos deployados como archivos `.pt` en Hetzner. **Razón:** Costo de GPU en Hetzner vs. Colab gratuito hace inviable entrenar en servidor. **Protocolo:** Modelo entrenado → `ml/models/` en repo → CI/CD auto-despliega.

ADR-004: MinIO para almacenamiento de imágenes (no AWS S3)

Decisión: MinIO self-hosted en Hetzner. **Razón:** Imágenes de cultivo son datos sensibles del cliente. Hetzner tiene costos predecibles. S3 tendría costos variables con alto volumen de imágenes de campo. **Migración futura:** MinIO es 100% compatible con la API de S3; migrar a S3 solo requiere cambiar variables de entorno.

ADR-005: Multi-tenant por schema vs. por base de datos

Decisión: Multi-tenant por `tenant_id` en cada tabla (shared schema). **Razón:** Para < 50 clientes, un schema compartido con row-level security es suficiente y más fácil de mantener. Schema por tenant introduce overhead de migrations.

Anti-Patrones a Evitar

Anti-patrón	Riesgo	Mitigación
Empezar Step 8 sin datos reales	Modelo con >40% MAPE, inútil para el cliente	Esperar 30 días de datos acumulados en producción
YOLOv8l o xl en Hetzner sin GPU	Latencia >5 seg por foto	Mantener YOLOv8n; benchmarking antes de upgradear
Saltarse PWA e ir directo a React Native	2x costo de desarrollo	PWA → Capacitor cubre el 95% del caso de uso con el 20% del esfuerzo
Un solo proceso Python para todo	Crash de YOLOv8 tira el API completo	Workers separados por servicio con <code>--workers 2</code> en Gunicorn
SQLite en producción	No soporta concurrencia multi-tenant	PostgreSQL desde el Step 2, nunca SQLite
Modelos directamente en repo git	Archivos .pt >50MB rompen el repo	Modelos en MinIO + solo el path en <code>model_registry</code> tabla

Protocolo de Mutación del Plan

Si necesitas modificar este plan:

- **Insertar step:** Numerarlo X.5 (ej. Step 3.5), actualizar el grafo de dependencias
- **Saltar step:** Documentar razón + qué asunciones cambian para steps dependientes
- **Reordenar:** Verificar que no se rompen dependencias en el grafo
- **Abandonar step:** Documentar en `plans/CHANGELOG.md` con fecha y razón

Archivos de Referencia

Archivo	Descripción
business-model-canvas.md	Segmentos, propuesta de valor, precios
strategy-canvas.md	Blue Ocean vs. GHT Corp, Agerpix, EOSDA
product-strategy-report.md	Roadmap NOW/NEXT/LATER, user stories, métricas
lean-canvas.md	Hipótesis críticas y experimentos de validación
piloto/RESUMEN_DATASET_FINAL.md	Estado del dataset de clavel (312 imgs)
proyecto_conteo_cajas/README_QUICK_START.md	Modelo cajas v2 (97.45% mAP50)
serial_reader_app/README.md	Implementación QR/barcode actual

Blueprint generado con la skill `blueprint` v1.0 — Marzo 2026 AgriVision Platform — Sabana Norte de Bogotá, Colombia